

可信开源软件大作业报告

开源 python 软件仓库分析工具 设计与实现

学院 (学部): 软件学院

专 业: 软件工程

学 生 姓 名: 王硕 赵浩然 彭博 田红森 赵家乐

学 号: 20232241155 20232241289

20232241037 20231011028

20232241417

指 导 教 师: _____

评 阅 教 师: _____

完 成 日 期: 2026 年 2 月

大连理工大学

Dalian University of Technology

目 录

介绍.....	1
0.1 系统架构与实现.....	1
0.1.1 主控制模块 (main.py).....	1
0.1.2 Git 分析模块.....	2
0.1.3 代码质量分析模块	2
0.1.4 GitHub 生态分析模块.....	2
0.2 技术特点.....	2
1 Git 仓库分析	3
1.1 基于 GitGraph 的 Git 提交历史可视化.....	3
1.1.1 核心实现	3
1.2 多维度 Git 仓库分析.....	4
1.2.1 实现技术与工具.....	4
1.2.2 核心分析函数实现	4
1.2.3 分析维度详解	6
1.2.3.1 开发活跃度分析	6
1.2.3.2 贡献者画像分析	8
1.2.3.3 代码演进与质量分析	9
1.2.3.4 提交行为语义分析.....	11
2 代码静态/动态分析.....	15
2.1 代码复杂度分析.....	15
2.1.1 分析方法与实现.....	15
2.1.2 核心实现代码	15
2.1.3 可视化分析.....	16
2.2 项目依赖与安全审计.....	16
2.2.1 分析维度	16
2.2.2 结果展示	17
2.3 代码漏洞扫描 (SAST)	17
2.3.1 扫描机制	17
2.3.2 风险评估	18
2.4 代码质量与动态分析.....	18

3 深度代码分析.....	20
3.1 代码复杂度分析.....	20
3.1.1 分析方法与实现.....	20
3.1.2 分析结果	20
3.2 项目依赖与安全审计.....	20
3.2.1 分析维度	20
3.2.2 核心实现	20
3.2.3 分析结果	22
3.3 代码漏洞扫描 (SAST)	22
3.3.1 扫描机制	22
3.3.2 实现机制	22
3.3.3 风险评估	24
3.4 GitHub 生态分析	25
3.4.1 安全 Issues 分析.....	25
3.4.2 PR 分析.....	27
3.4.3 GitHub Actions 工作流分析.....	29
3.5 综合评估.....	31
结 论.....	33
附录 A 原始分析数据摘录.....	34

介绍

随着开源软件的快速发展，软件仓库中积累了大量开发数据与协作信息。这些数据蕴含着丰富的项目质量、团队协作及代码演进信息，如何高效分析并挖掘这些信息，是提升软件质量与开发效率的核心问题。

本项目设计并实现了一款多维度开源 Python 仓库分析器，专门针对 COMTool 项目开展分析工作。该分析器从三个核心维度对软件仓库进行全面剖析：

该分析器从三个核心维度对软件仓库进行全面剖析：在 Git 仓库分析方面，通过解析提交历史、开发者贡献、代码演进等数据，梳理项目开发轨迹与团队结构特征；在代码静态/动态分析方面，整合多款工具完成代码复杂度评估、依赖安全审计、漏洞扫描及测试覆盖率分析；在 GitHub 仓库分析方面，基于 GitHub API 深度分析 PR 协作流程、Issue 追踪机制与 Actions CI/CD 工作流。

0.1 系统架构与实现

本分析器采用模块化架构设计，核心模块如下：

0.1.1 主控制模块 (main.py)

主模块负责协调整个分析流程，涵盖环境初始化、仓库克隆、数据收集及各分析模块调度，核心流程如下：

```
def main():  
    # 环境初始化  
    setup_environment()  
  
    # 克隆/加载仓库  
    repo = clone_repo(GIT_URL, REPO_PATH)  
  
    # 获取提交历史  
    commits = get_git_history(repo, MAX_COMMITS)  
  
    # 执行基础Git分析  
    analyze.run_all_analysis(repo, commits, output_dir=str(REPORT_DIR), prefix=PREFIX)  
  
    # 执行深度代码分析
```

`run_advanced_analysis()`

0.1.2 Git 分析模块

实现多维度 Git 仓库分析功能：实现提交历史可视化（基于 gitgraph.js）、开发者活跃度统计、代码演进趋势分析以及关键词语义分析等功能。

0.1.3 代码质量分析模块

集成多款主流静态分析工具：集成 Radon 进行代码复杂度分析、Bandit 进行安全漏洞扫描、Pylint 进行代码规范检查以及 Safety 进行依赖安全审计。

0.1.4 GitHub 生态分析模块

基于 PyGithub 库实现以下功能：实现 PR 协作效率分析、Issue 安全风险评估以及 Actions 工作流安全检测。

0.2 技术特点

具有高自动化程度，覆盖从数据收集到报告生成的全流程自动化处理；提供丰富的可视化展示，包括多类型图表与交互式展示形式；采用模块化设计，具备良好的扩展性与定制性；同时具备跨平台兼容性，适配 Windows、macOS、Linux 等主流操作系统。

本次分析旨在为 COMTool 项目提供全面的质量评估，识别潜在问题点，并为后续代码优化与安全加固工作提供数据支撑。

报告整体结构如下：第 1 章介绍 Git 仓库多维度分析方法；第 2 章详细阐述代码静态与动态分析技术；第 3 章展示深度代码分析结果；第 4 章总结分析发现并提出改进建议。

1 Git 仓库分析

1.1 基于 GitGraph 的 Git 提交历史可视化

本分析工具内置 `html_generator.py` 模块，用于生成 Git 提交历史可视化报告。该模块读取 Git 仓库提交日志（含哈希值、作者、日期、提交信息及父提交关系），并将结构化数据注入 HTML 模板，输出交互式可视化页面。

1.1.1 核心实现

`html_generator.py` 核心实现如下：

```
def generate_git_tree_html(commits, repo_url):
    """生成Git提交历史HTML可视化页面"""
    commits_data = []
    for commit in commits:
        commit_info = {
            'hash': commit['hash'],
            'hashFull': commit['hashFull'],
            'author': commit['author'],
            'date': commit['date'],
            'message': commit['message'],
            'parents': commit['parents']
        }
        commits_data.append(commit_info)

    # 渲染模板并保存文件
    html_content = render_template('git_tree_template.html',
                                   commits=commits_data,
                                   repo_url=repo_url)

    with open('git_tree.html', 'w', encoding='utf-8') as f:
        f.write(html_content)

    return 'git_tree.html'
```

前端可视化采用 `gitgraph.js` 开源库，可直观展示 Git 分支结构、合并历史及提交节点。生成的 `git_tree.html` 支持交互式浏览，用户可查看项目演进路线、分支合并情况及开发者提交记录。

结果如图1.1所示，展示了 COMTool 项目 Git 提交历史。每个节点代表一次提交，节点连线表示父子关系，不同颜色区分分支/标签，鼠标悬停可查看提交详情。



图 1.1 COMTool 项目 Git 提交历史可视化

1.2 多维度 Git 仓库分析

本节详细介绍 `analyze.py` 模块实现的多维度 Git 仓库分析功能，通过量化数据揭示项目开发节奏、团队结构及代码演进规律。

1.2.1 实现技术与工具

分析工具基于 Python 开发，核心依赖以下开源库：GitPython 用于与本地 Git 仓库交互，遍历提交历史、提取提交哈希/作者/时间戳/文件变更统计等元数据；Matplotlib 作为可视化引擎，配置中文字体（SimHei/Source Han Sans CN）确保中文标签正常显示；Pandas 用于高效处理时间序列数据，支持数据聚合、重采样与统计分析；Collections（Counter）则用于快速统计作者提交次数、关键词频率等数据。

1.2.2 核心分析函数实现

`analyze.py` 关键分析函数如下：

```
def analyze_authors(commits):
    """分析开发者贡献"""
    author_count = Counter()
    for commit in commits:
        author = commit.get('author', 'Unknown')
        author_count[author] += 1
    return author_count.most_common(10)

def analyze_monthly_activity(commits):
    """分析月度活跃度"""
    monthly_data = defaultdict(int)
    for commit in commits:
        date_str = commit.get('date', '')
        if date_str:
            month = date_str[:7] # YYYY-MM
            monthly_data[month] += 1
    return dict(sorted(monthly_data.items()))

def analyze_keywords(commits):
    """分析提交信息关键词"""
    keywords = {
        'add': ['add', '新增', '增加', 'create', 'new'],
        'fix': ['fix', '修复', 'bug', '解决'],
        'update': ['update', '更新', 'upgrade', '优化'],
        'refactor': ['refactor', '重构', '优化', '改进'],
        'remove': ['remove', '删除', 'del', 'rm']
    }

    keyword_count = Counter()
    for commit in commits:
        message = commit.get('message', '').lower()
        for category, words in keywords.items():
            if any(word in message for word in words):
```

```
keyword_count[category] += 1
return dict(keyword_count)
```

1.2.3 分析维度详解

1.2.3.1 开发活跃度分析

开发活跃度是衡量项目生命周期的核心指标，工具从宏观月度趋势和微观时段分布两个维度分析。

月度活跃度趋势：提取提交日期并按月份统计提交量，可清晰反映项目起步期、爆发期及平稳维护期。

实现机制：

```
def plot_monthly_activity(monthly_data, output_path):
    """绘制月度活跃度图表"""
    plt.figure(figsize=(12, 6))
    months = list(monthly_data.keys())
    counts = list(monthly_data.values())

    plt.plot(months, counts, marker='o', linewidth=2, markersize=6)
    plt.title('项目月度提交活跃度趋势', fontsize=16, fontweight='bold')
    plt.xlabel('月份', fontsize=12)
    plt.ylabel('提交数量', fontsize=12)
    plt.xticks(rotation=45)
    plt.grid(True, alpha=0.3)
    plt.tight_layout()

    plt.savefig(output_path, dpi=300, bbox_inches='tight')
    plt.close()
```

全天时段提交分布：提取提交时间的小时属性，分析开发者工作习惯，反映主力团队工作时区及加班情况。

实现机制：

```
def analyze_hourly_distribution(commits):
    """分析全天时段提交分布"""
    hourly_count = Counter()
```

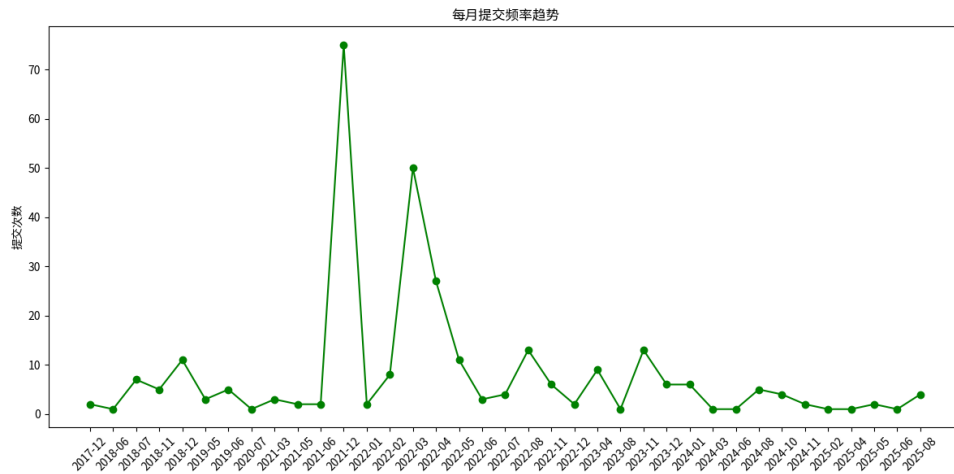


图 1.2 项目月度提交活跃度趋势

```
for commit in commits:
    date_str = commit.get('date', '')
    if date_str and len(date_str) > 13:
        hour = int(date_str[11:13])
        hourly_count[hour] += 1

# 补全0-23小时数据
for hour in range(24):
    hourly_count[hour] = hourly_count.get(hour, 0)
return dict(sorted(hourly_count.items()))
```

```
def plot_hourly_activity(hourly_data, output_path):
    """绘制全天时段提交分布图"""
    plt.figure(figsize=(12, 6))
    hours = list(hourly_data.keys())
    counts = list(hourly_data.values())

    plt.bar(hours, counts, color='skyblue', edgecolor='navy', alpha=0.7)
    plt.title('全天各时段提交频率分布', fontsize=16, fontweight='bold')
    plt.xlabel('小时 (24小时制)', fontsize=12)
    plt.ylabel('提交数量', fontsize=12)
```

```
plt.xticks(range(0, 24))
plt.grid(True, alpha=0.3)
plt.tight_layout()

plt.savefig(output_path, dpi=300, bbox_inches='tight')
plt.close()
```

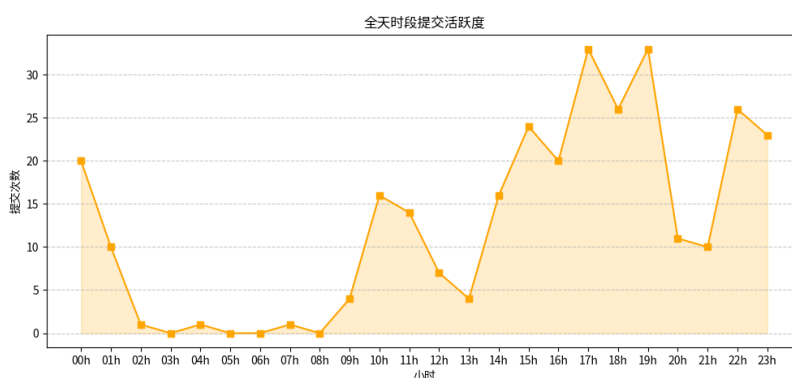


图 1.3 全天各时段提交频率分布

1.2.3.2 贡献者画像分析

清洗并统计 Git 提交的 author 字段，识别核心维护者及社区参与度。

核心贡献者统计：统计作者提交次数并生成前 10 名条形图，符合“二八定律”（少数核心成员贡献大部分代码）。

实现机制：

```
def plot_author_contributions(author_data, output_path):
    """绘制核心贡献者统计图"""
    plt.figure(figsize=(10, 6))
    authors = [item[0] for item in author_data]
    counts = [item[1] for item in author_data]

    bars = plt.barh(authors, counts, color='lightcoral', edgecolor='darkred')
    plt.title('前10名作者提交贡献统计', fontsize=16, fontweight='bold')
    plt.xlabel('提交数量', fontsize=12)
    plt.ylabel('作者', fontsize=12)
```

```

# 添加数值标签
for bar, count in zip(bars, counts):
    plt.text(bar.get_width() + max(counts)*0.01,
             bar.get_y() + bar.get_height()/2,
             str(count), ha='left', va='center', fontsize=10)

plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(output_path, dpi=300, bbox_inches='tight')
plt.close()

```

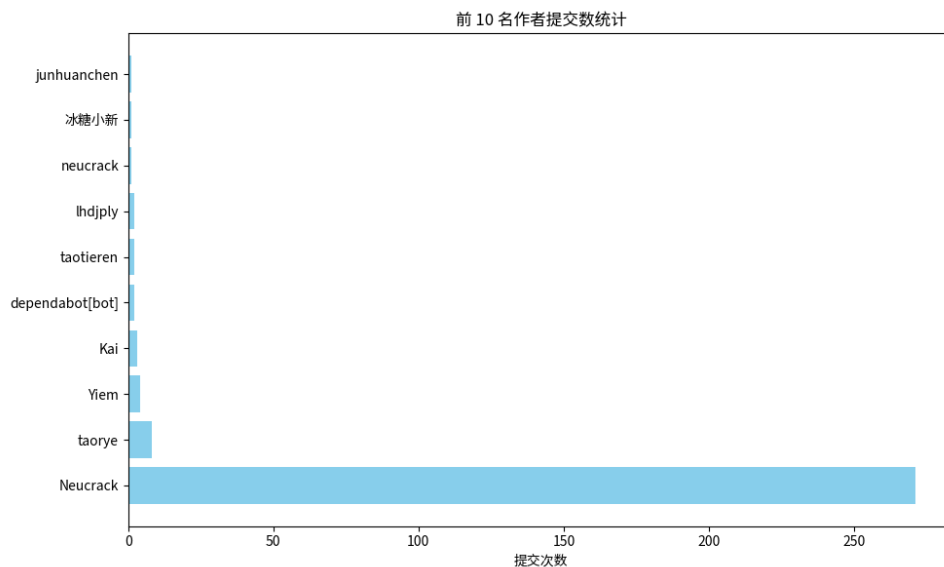


图 1.4 前 10 名作者提交贡献统计

1.2.3.3 代码演进与质量分析

代码量变化直观反映软件成长过程，工具从净行数演变和增删趋势两方面分析。

代码行数 (LOC) 演变：累加每次提交的新增/删除行数，计算净行数累计曲线，陡峭上升对应新功能开发，平缓/下降对应重构或清理死代码。

实现机制：

```

def analyze_loc_growth(commits):
    """分析代码行数演变"""

```

```
loc_data = []
total_loc = 0
for commit in commits:
    insertions = commit.get('stats', {}).get('insertions', 0)
    deletions = commit.get('stats', {}).get('deletions', 0)
    net_change = insertions - deletions
    total_loc += net_change

    loc_data.append({
        'date': commit.get('date', ''),
        'insertions': insertions,
        'deletions': deletions,
        'net_change': net_change,
        'total_loc': total_loc
    })
return loc_data

def plot_loc_growth(loc_data, output_path):
    """绘制代码行数演变图"""
    plt.figure(figsize=(12, 6))
    dates = [item['date'] for item in loc_data]
    total_loc = [item['total_loc'] for item in loc_data]

    plt.plot(dates, total_loc, linewidth=2, color='green')
    plt.title('代码库净行数演变趋势', fontsize=16, fontweight='bold')
    plt.xlabel('时间', fontsize=12)
    plt.ylabel('累计代码行数', fontsize=12)
    plt.xticks(rotation=45)
    plt.grid(True, alpha=0.3)
    plt.tight_layout()

    plt.savefig(output_path, dpi=300, bbox_inches='tight')
    plt.close()
```

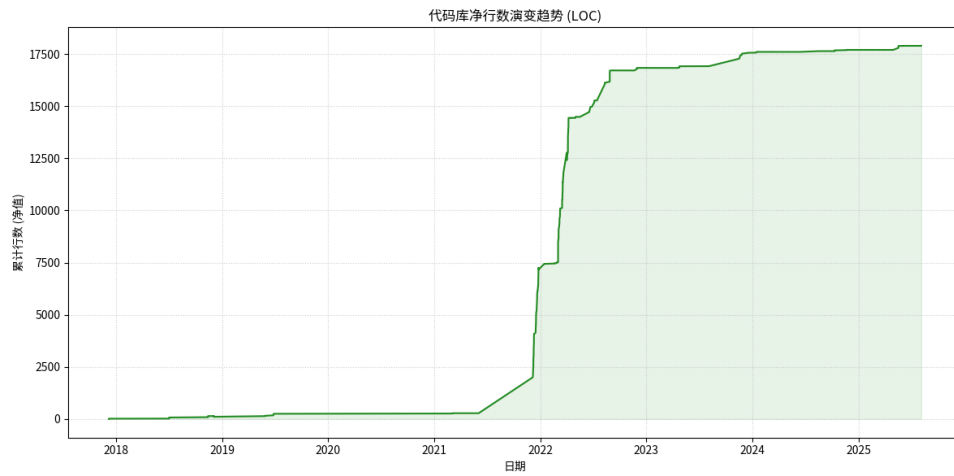


图 1.5 代码库净行数演变趋势

此外，工具分析的**代码增删趋势**（图1.6）可展示新增/删除行数波动，删除行数显著升高通常标志架构调整或技术债务偿还。

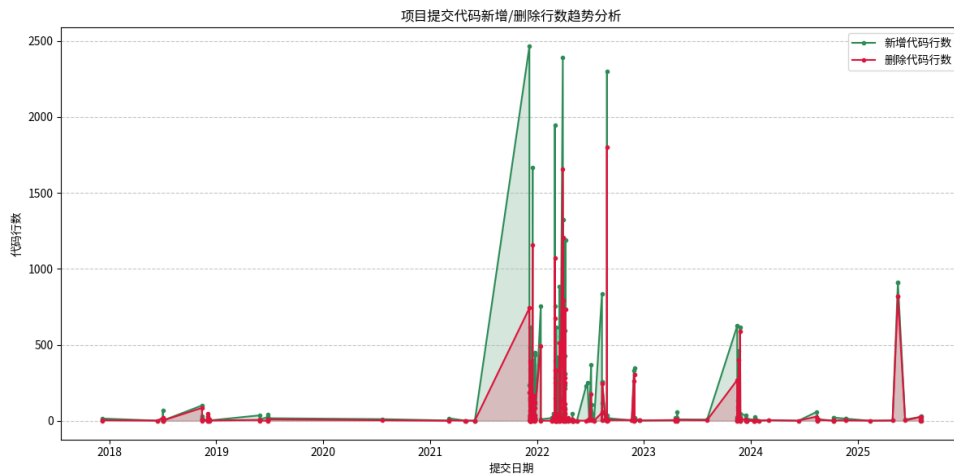


图 1.6 代码新增与删除行数趋势对照

1.2.3.4 提交行为语义分析

对 Commit Message 进行文本挖掘，理解开发者核心工作内容。

关键词分布分析：针对中文开发场景定义关键词映射，通过模式匹配生成分布饼图，直观判断项目阶段（新增占比高 = 迭代期，修复占比高 = 维护期）。

实现机制：

```
def plot_keywords_distribution(keyword_data, output_path):
```

```
"""绘制关键词分布饼图"""
plt.figure(figsize=(8, 8))
labels = list(keyword_data.keys())
sizes = list(keyword_data.values())

# 中文标签映射
label_map = {
    'add': '新增/Add',
    'fix': '修复/Fix',
    'update': '更新/Update',
    'refactor': '重构/Refactor',
    'remove': '删除/Remove'
}
display_labels = [label_map.get(label, label) for label in labels]

colors = ['#ff9999', '#66b3ff', '#99ff99', '#ffcc99', '#ff99cc']
plt.pie(sizes, labels=display_labels, autopct='%1.1f%%',
        startangle=90, colors=colors)
plt.title('提交信息高频关键词分布', fontsize=16, fontweight='bold')
plt.axis('equal')

plt.savefig(output_path, dpi=300, bbox_inches='tight')
plt.close()
```

修改热点分析：统计提交涉及的文件路径，识别修改频率最高的目录/模块，这些热点区域是业务核心且易出 Bug，需重点关注。

实现机制：

```
def analyze_file_hotspots(commits):
    """分析文件修改热点"""
    file_count = Counter()
    for commit in commits:
        modified_files = commit.get('files', [])
        for file_path in modified_files:
```

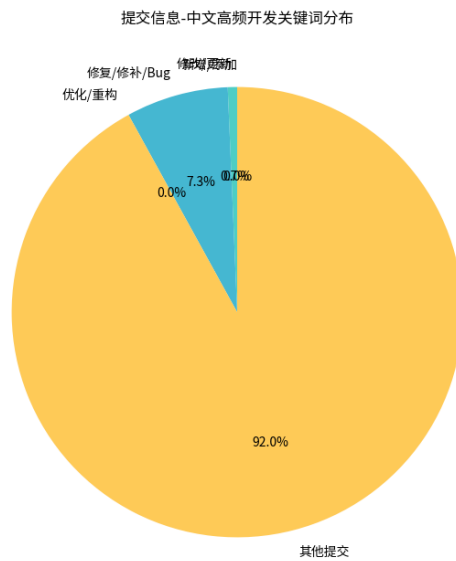


图 1.7 提交信息高频关键词分布

```
dir_path = os.path.dirname(file_path)
if dir_path:
    file_count[dir_path] += 1
return file_count.most_common(10)

def plot_file_hotspots(hotspot_data, output_path):
    """绘制修改热点分析图"""
    plt.figure(figsize=(10, 6))
    directories = [item[0] for item in hotspot_data]
    counts = [item[1] for item in hotspot_data]

    bars = plt.barh(directories, counts, color='lightblue', edgecolor='navy')
    plt.title('最常被修改的代码目录（热点分析）', fontsize=16, fontweight='bold')
    plt.xlabel('修改次数', fontsize=12)
    plt.ylabel('目录路径', fontsize=12)
```

```
# 添加数值标签
for bar, count in zip(bars, counts):
    plt.text(bar.get_width() + max(counts)*0.01,
             bar.get_y() + bar.get_height()/2,
             str(count), ha='left', va='center', fontsize=10)

plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(output_path, dpi=300, bbox_inches='tight')
plt.close()
```

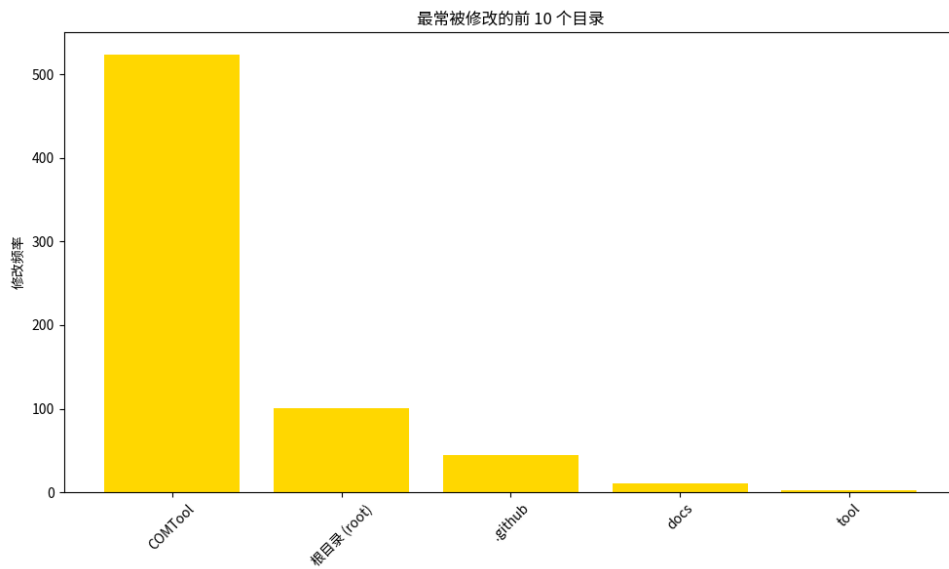


图 1.8 最常被修改的代码目录 (热点分析)

2 代码静态/动态分析

本章详细介绍针对代码库开展的深度质量分析，涵盖代码复杂度评估、项目依赖安全审计、静态漏洞扫描及动态测试覆盖率分析四个维度，全方位评估软件鲁棒性与可维护性。

2.1 代码复杂度分析

代码复杂度是衡量软件可维护性的核心指标，本项目基于 `radon` 库，通过 `AdvancedRadonAnalyzer` 模块实现自动化复杂度计算与可视化分析。

2.1.1 分析方法与实现

模块遍历仓库中所有 Python 文件，基于抽象语法树 (AST) 计算以下核心指标：圈复杂度 (Cyclomatic Complexity, CC) 用于统计代码线性独立路径数量衡量逻辑复杂性，通过 `radon.complexity.cc_visit` 解析代码块，统计 `if/while/for` 等控制流结构；可维护性指数 (Maintainability Index, MI) 则利用 `radon.metrics.mi_visit` 计算 0-100 分的综合评分，结合圈复杂度、代码行数、Halstead 体积和注释比例。

2.1.2 核心实现代码

`AdvancedRadonAnalyzer` 类核心实现如下：

```
class AdvancedRadonAnalyzer:
    def __init__(self, repo_path, output_dir):
        self.repo_path = Path(repo_path)
        self.output_dir = Path(output_dir)

    def analyze_file(self, file_path):
        """分析单个文件复杂度"""
        with open(file_path, 'r', encoding='utf-8') as f:
            code = f.read()

        # 计算圈复杂度与可维护性指数
        cc_results = cc_visit(code)
        mi_result = mi_visit(code, True)
```

```
    return {
        'file': str(file_path),
        'cc_complexity': len(cc_results),
        'mi_score': mi_result,
        'loc': len(code.split('\n'))
    }

def batch_analyze(self):
    """批量分析所有Python文件"""
    python_files = list(self.repo_path.rglob('*.py'))
    results = []

    for file_path in python_files:
        try:
            result = self.analyze_file(file_path)
            results.append(result)
        except Exception as e:
            print(f"分析文件 {file_path} 出错: {e}")

    return results
```

2.1.3 可视化分析

分析结果聚合后生成图3.1，上部展示有效文件平均圈复杂度，下部展示可维护性指数分布，可快速定位复杂度过高（CC 高）或难以维护（MI 低）的”坏味道”模块。

2.2 项目依赖与安全审计

针对软件开发中的”供应链安全”问题，dependency_analyzer.py 模块基于 pipdeptree 和 safety 工具实现深度依赖分析。

2.2.1 分析维度

依赖树解析通过调用 pipdeptree -json 命令解析环境依赖关系，构建完整依赖树，识别直接依赖及隐蔽的传递依赖；漏洞数据库比对则集成 safety 工具，将依赖版本与 CVE 漏洞数据库比对，自动标记存在安全风险的包版本。



图 2.1 代码圈复杂度与可维护性指数分布

2.2.2 结果展示

图3.2展示分析结果：左侧饼图反映项目依赖构成（直接依赖 vs 传递依赖），右侧直方图统计不同风险等级漏洞数量。

2.3 代码漏洞扫描 (SAST)

为发现代码层面潜在安全隐患，项目集成 `bandit` 作为静态应用程序安全测试 (SAST) 工具。

2.3.1 扫描机制

`vulnerability_scanner.py` 模块封装 `BanditManager`，不执行代码，仅将 Python 源码解析为 AST，通过插件匹配已知漏洞模式，包括硬编码敏感信息（密码、API Key）、弱加密算法（MD5、SHA1）使用、危险函数调用（`eval()`、`subprocess.Popen` 的 Shell 注入风险）以及不安全网络绑定（如绑定到 `0.0.0.0`）。

项目依赖分析报告

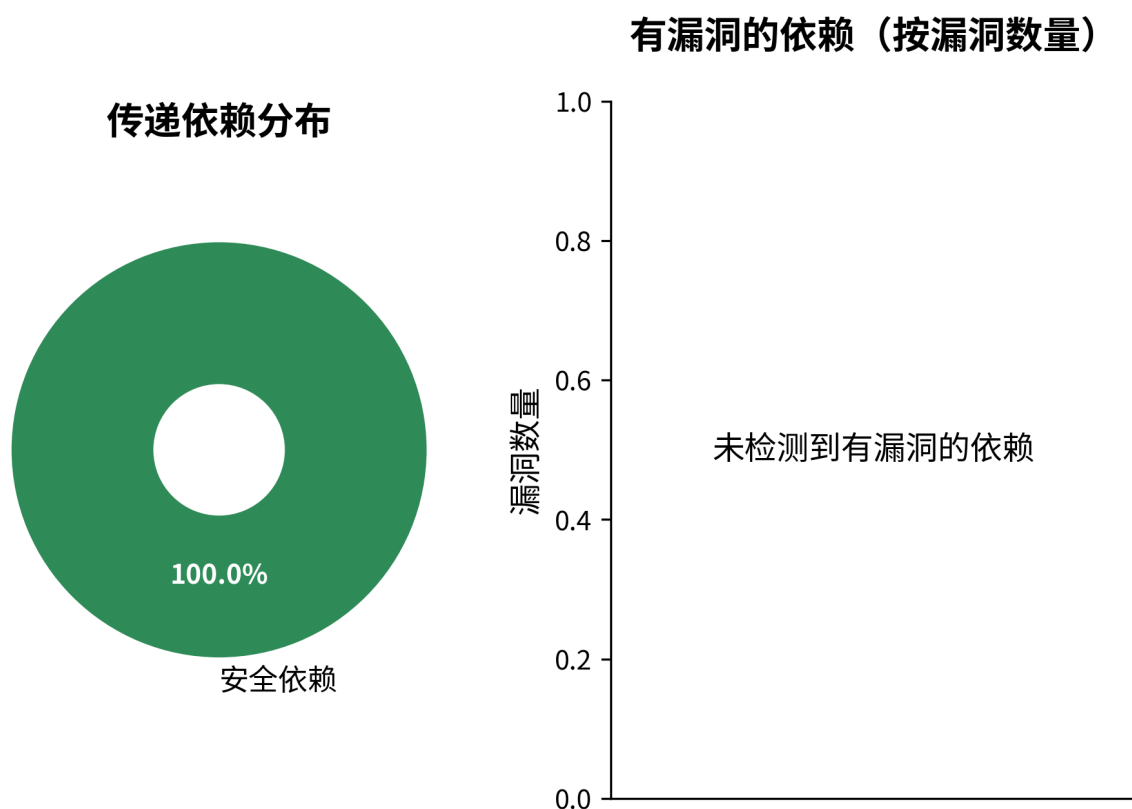


图 2.2 项目依赖构成与安全漏洞分析

2.3.2 风险评估

扫描结果分为 High、Medium、Low 三个风险等级，图3.3直观展示各类风险分布，为安全修复优先级提供数据支撑。

2.4 代码质量与动态分析

工具还整合 pylint 进行代码风格检查，以及 pytest-cov 开展动态测试覆盖率分析：Pylint 质量分布通过图2.4展示代码中规范(Convention)、重构(Refactor)、警告(Warning)、错误(Error)类问题占比；动态测试覆盖率则由 analyze.py 模块运行项目单元测试，生成覆盖率报告 (coverage_report.txt)，评估测试用例对代码逻辑的覆盖程度。

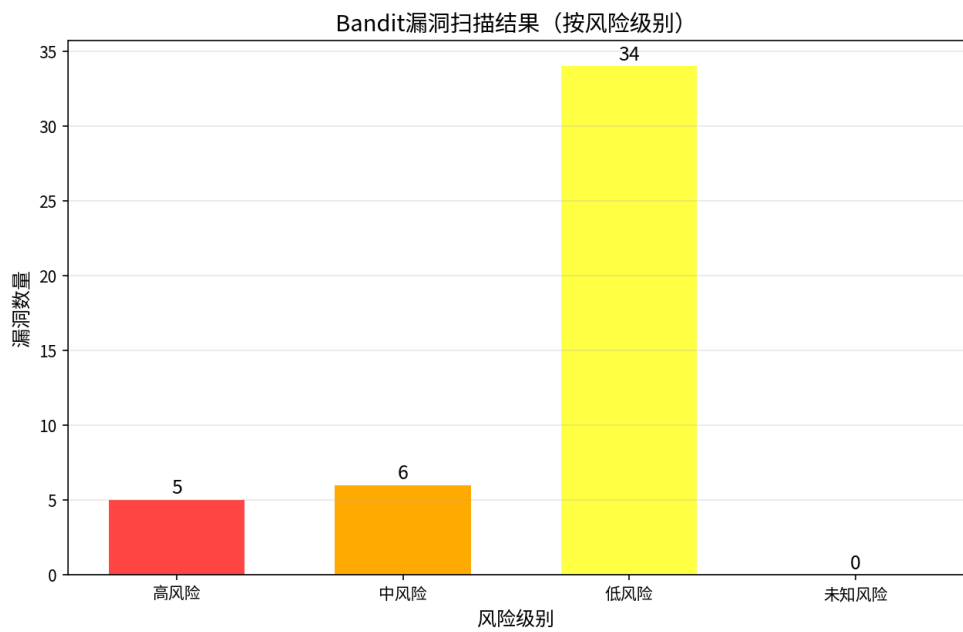


图 2.3 Bandit 静态漏洞扫描结果分布

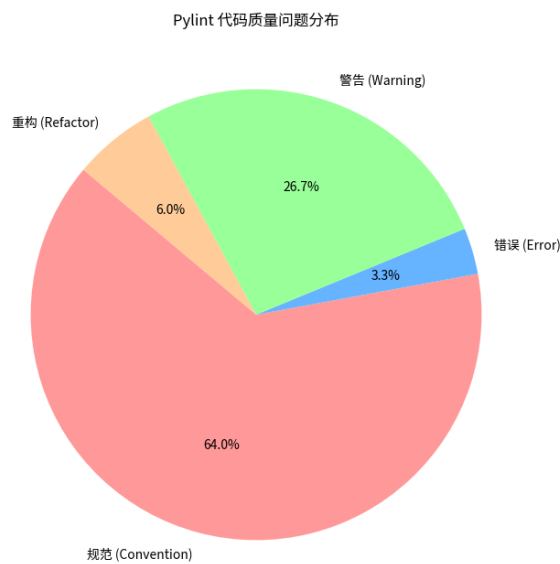


图 2.4 Pylint 代码质量问题分布

3 深度代码分析

本章详细介绍针对 COMTool 项目开展的深度代码分析，涵盖代码复杂度评估、项目依赖安全审计、静态漏洞扫描及 GitHub 生态分析四个维度，全方位评估软件鲁棒性与可维护性。

3.1 代码复杂度分析

代码复杂度是衡量软件可维护性的核心指标，本项目基于 `radon` 库，通过 `AdvancedRadonAnalyzer` 模块实现自动化复杂度计算与可视化分析。

3.1.1 分析方法与实现

模块遍历仓库中所有 Python 文件，基于抽象语法树 (AST) 计算以下核心指标：圈复杂度 (Cyclomatic Complexity, CC) 用于统计代码线性独立路径数量衡量逻辑复杂性，通过 `radon.complexity.cc_visit` 解析代码块，统计 `if/while/for` 等控制流结构；可维护性指数 (Maintainability Index, MI) 则利用 `radon.metrics.mi_visit` 计算 0-100 分综合评分，结合圈复杂度、代码行数、Halstead 体积和注释比例。

3.1.2 分析结果

分析覆盖 COMTool 项目 47 个文件 (有效文件 38 个)，图3.1展示代码复杂度分布。结果显示项目整体复杂度控制良好，大部分模块圈复杂度处于合理范围，代码结构清晰，维护成本较低。

3.2 项目依赖与安全审计

针对软件开发中的“供应链安全”问题，`DependencyAnalyzer` 模块基于 `pipdeptree` 和 `safety` 工具实现深度依赖分析。

3.2.1 分析维度

依赖树解析通过调用 `pipdeptree -json` 命令解析环境依赖关系，构建完整依赖树，识别直接依赖及隐蔽的传递依赖；漏洞数据库比对则集成 `safety` 工具，将依赖版本与 CVE 漏洞数据库比对，自动标记存在安全风险的包版本。

3.2.2 核心实现

`DependencyAnalyzer` 关键实现如下：

```
class DependencyAnalyzer:
```

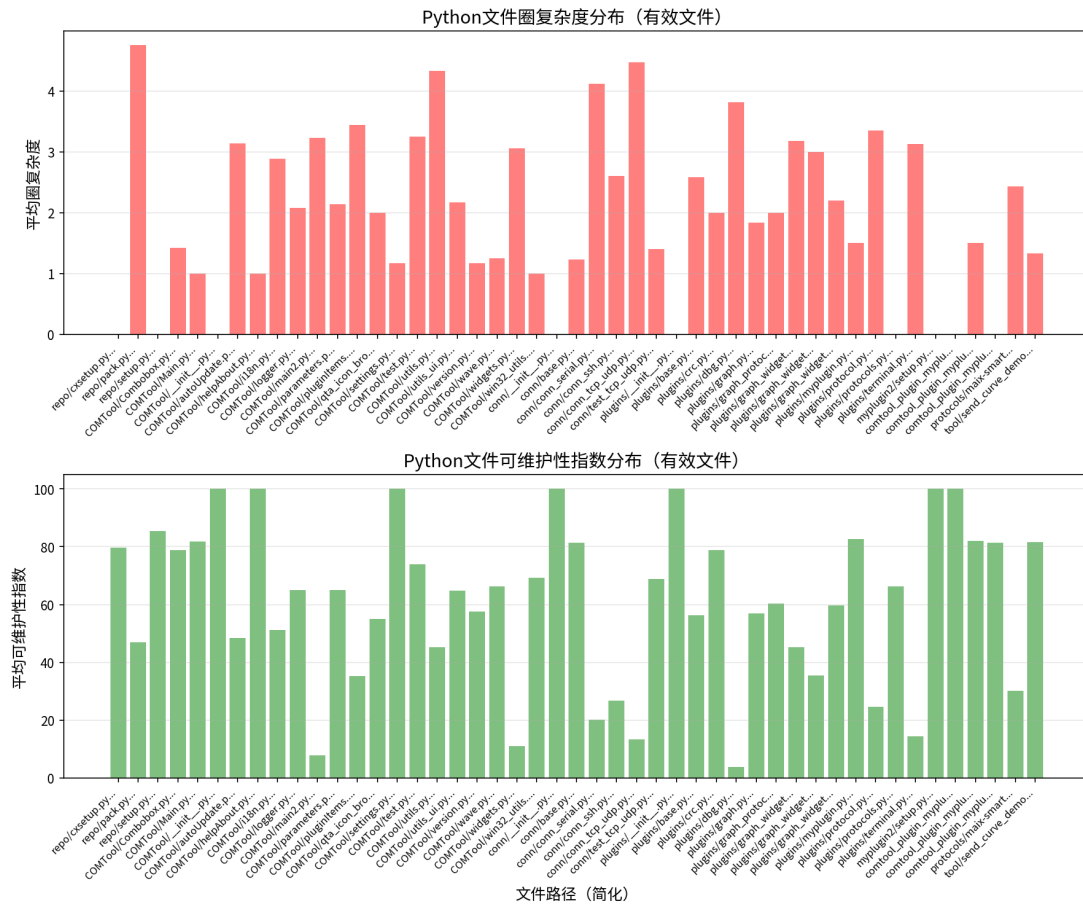


图 3.1 代码圈复杂度与可维护性指数分布

```
def analyze_python_deps(self):
    """分析Python依赖"""
    # 获取依赖树
    result = subprocess.run(
        ['pipdeptree', '--json'],
        capture_output=True, text=True
    )

    if result.returncode == 0:
        deps = json.loads(result.stdout)
        return self._process_dependencies(deps)
    else:
        raise RuntimeError("依赖分析失败")
```

```
def _check_vulnerabilities(self, deps):
    """检查依赖漏洞"""
    result = subprocess.run(
        ['safety', 'check', '--json'],
        capture_output=True, text=True
    )

    vulnerabilities = []
    if result.returncode != 0:
        try:
            vulnerabilities = json.loads(result.stdout)
        except:
            pass

    return vulnerabilities
```

3.2.3 分析结果

图3.2展示分析结果：项目依赖管理规范，**未检测到含漏洞的依赖包**，表明项目在依赖选择与维护方面表现良好，有效降低供应链攻击风险。

3.3 代码漏洞扫描 (SAST)

为发现代码层面潜在安全隐患，项目集成 `bandit` 作为静态应用程序安全测试 (SAST) 工具。

3.3.1 扫描机制

`AdvancedVulnerabilityScanner` 模块封装 `BanditManager`，不执行代码，仅将 Python 源码解析为 AST，通过插件匹配已知漏洞模式，包括硬编码敏感信息（密码、API Key）、弱加密算法（MD5、SHA1）使用、危险函数调用（`eval()`、`subprocess.Popen` 的 Shell 注入风险）以及不安全网络绑定（如绑定到 0.0.0.0）。

3.3.2 实现机制

漏洞扫描核心实现如下：

```
class AdvancedVulnerabilityScanner:
```

项目依赖分析报告

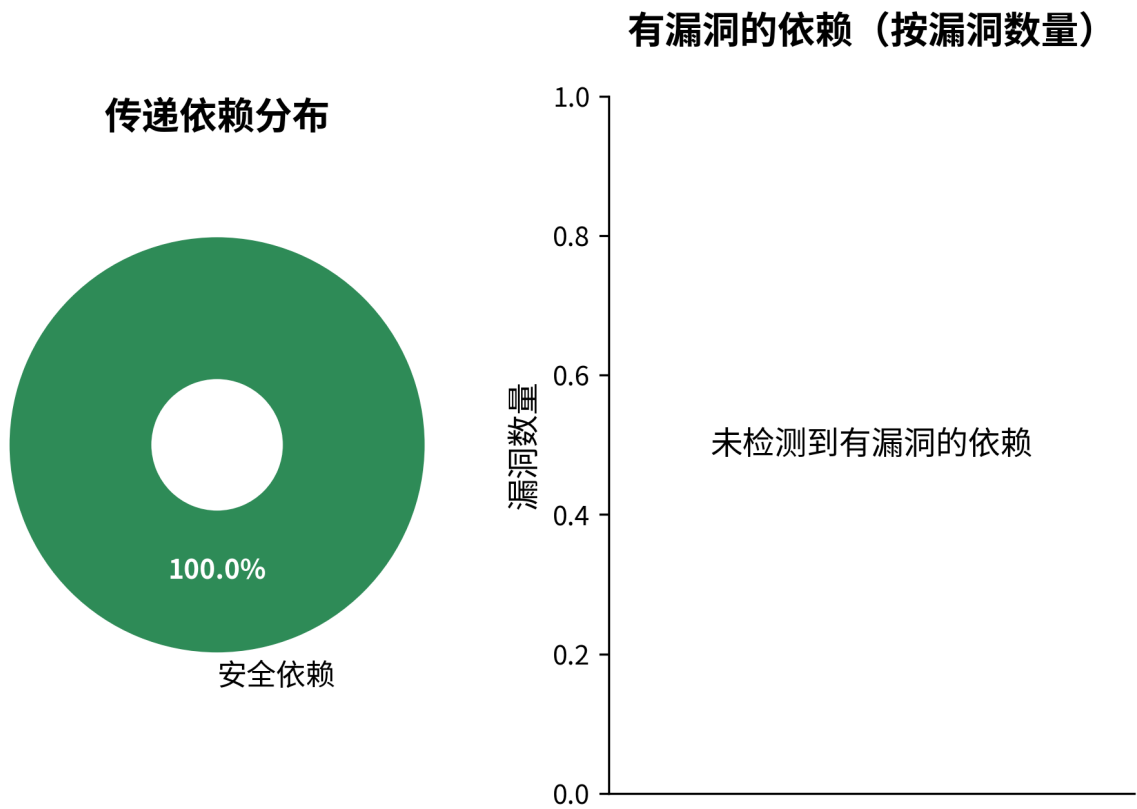


图 3.2 项目依赖构成与安全漏洞分析

```
def __init__(self, repo_path, output_dir):
    self.repo_path = Path(repo_path)
    self.output_dir = Path(output_dir)

def advanced_bandit_scan(self):
    """执行Bandit漏洞扫描"""
    # 配置扫描参数
    cmd = [
        'bandit', '-r', str(self.repo_path),
        '-f', 'json', '-o', str(self.output_dir / 'bandit_report.json')
    ]
```

```
result = subprocess.run(cmd, capture_output=True, text=True)

# 解析结果（返回1表示发现问题）
if result.returncode in [0, 1]:
    return self._parse_bandit_results()
else:
    raise RuntimeError(f"Bandit扫描失败: {result.stderr}")

def _parse_bandit_results(self):
    """解析扫描结果"""
    report_file = self.output_dir / 'bandit_report.json'
    if report_file.exists():
        with open(report_file, 'r') as f:
            data = json.load(f)

        # 按严重程度统计
        severity_count = {'high': 0, 'medium': 0, 'low': 0}
        for issue in data.get('results', []):
            severity = issue.get('issue_severity', 'low')
            if severity in severity_count:
                severity_count[severity] += 1

        return {
            'total_issues': len(data.get('results', [])),
            'severity_count': severity_count,
            'results': data.get('results', [])
        }

    return {'total_issues': 0, 'severity_count': {'high': 0, 'medium': 0, 'low': 0}}
```

3.3.3 风险评估

扫描结果按 High、Medium、Low 三个等级分类：高风险漏洞 5 个，中风险漏洞 6 个，低风险漏洞 34 个。

图3.3展示风险分布，漏洞主要集中在中低风险等级，高风险漏洞数量较少，为安全修复优先级提供明确数据支撑。

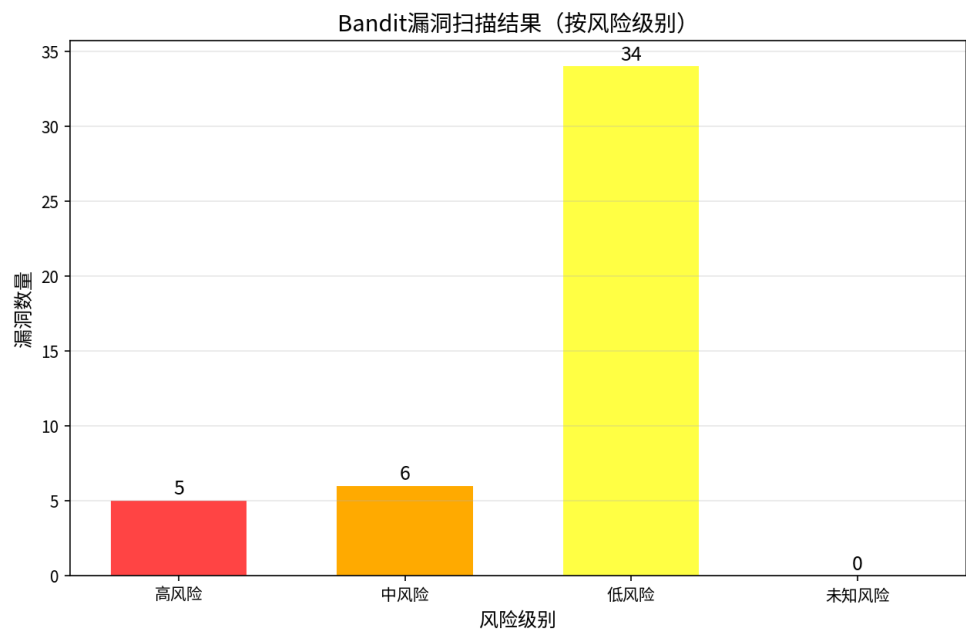


图 3.3 Bandit 静态漏洞扫描结果分布

3.4 GitHub 生态分析

3.4.1 安全 Issues 分析

通过 GitHubIssueAnalyzer 模块分析 Neutree/COMTool 仓库近 180 天 issues 数据，模块基于 PyGithub 库实现，核心功能如下：

```
class GitHubIssueAnalyzer:
    def __init__(self, repo_owner, repo_name, output_dir, token=None):
        self.repo_owner = repo_owner
        self.repo_name = repo_name
        self.output_dir = Path(output_dir)
        self.github = Github(token) if token else Github()
        self.repo = self.github.get_repo(f"{repo_owner}/{repo_name}")

    def get_issues(self, days_back=180):
        """获取指定时间范围的issues"""
```

```
since = datetime.now() - timedelta(days=days_back)
issues = []

for issue in self.repo.get_issues(state='all', since=since):
    issues.append({
        'number': issue.number,
        'title': issue.title,
        'body': issue.body or "",
        'state': issue.state,
        'created_at': issue.created_at,
        'labels': [label.name for label in issue.labels]
    })

return issues

def analyze_security_issues(self, issues):
    """分析安全相关issues"""
    security_keywords = [
        'vulnerability', 'security', 'cve', 'exploit', 'injection',
        'xss', 'rce', 'overflow', 'bypass', '权限', '漏洞', '安全'
    ]

    security_issues = []
    for issue in issues:
        text = f"{issue['title']} {issue['body']}".lower()
        if any(keyword.lower() in text for keyword in security_keywords):
            security_issues.append(issue)

    return {
        'security_issues': security_issues,
        'severity_count': self._classify_severity(security_issues)
    }
```

分析结果：总计获取 issues 9 个，其中安全相关 issues 1 个，风险等级分布为高风

险 1 个，中/低风险 0 个。

项目存在 1 个需重点关注的安全问题，建议开发团队优先处理该高风险 issue。

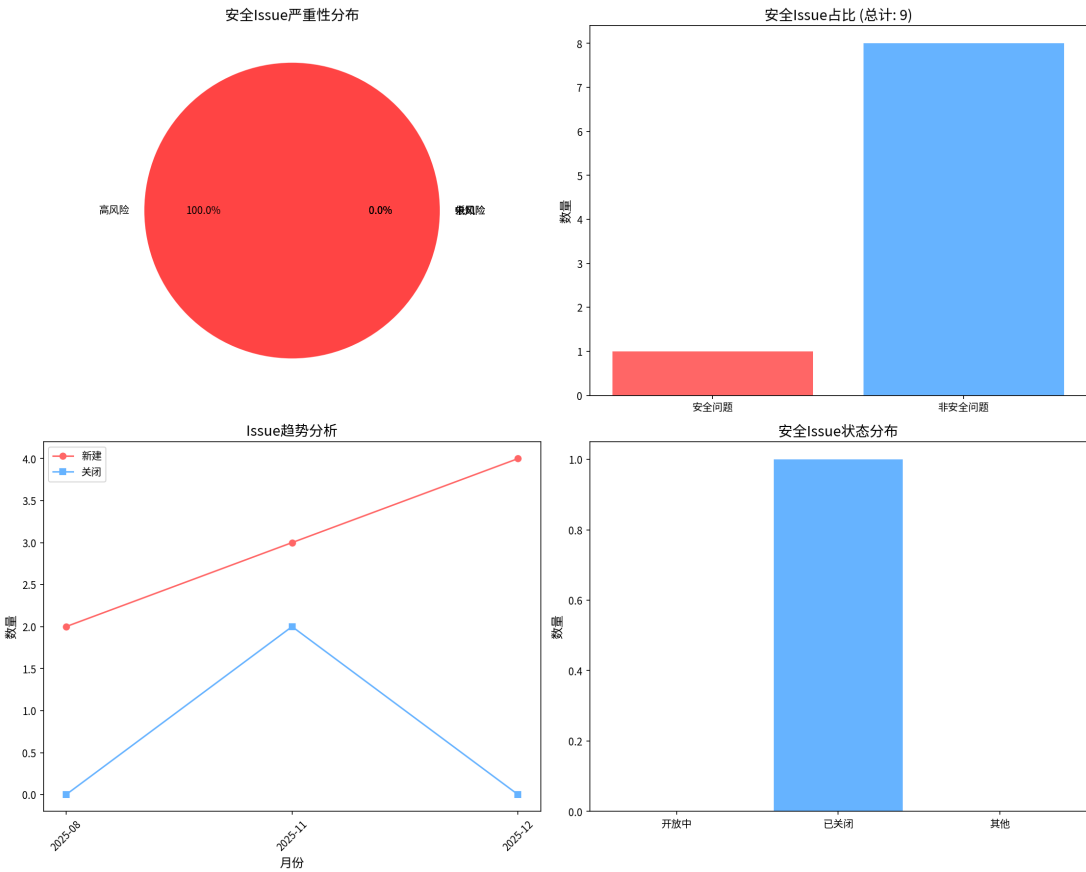


图 3.4 GitHub 安全 Issues 分析结果

3.4.2 PR 分析

通过 analyze_pr_repository 函数分析项目 PR 数据，实现 PR 协作效率全面评估：

```
def analyze_pr_repository(repo_full_name, days_threshold=7, github_token=None):  
    """分析GitHub PR数据"""  
    g = Github(github_token) if github_token else Github()  
    repo = g.get_repo(repo_full_name)  
  
    pr_data = []  
    for pr in repo.get_pulls(state='all'):
```

```
pr_info = {
    'number': pr.number,
    'title': pr.title,
    'state': pr.state,
    'created_at': pr.created_at,
    'merged_at': pr.merged_at,
    'closed_at': pr.closed_at,
    'review_comments': pr.review_comments,
    'comments': pr.comments,
    'additions': pr.additions,
    'deletions': pr.deletions
}
pr_data.append(pr_info)

# 计算核心指标
total_pr = len(pr_data)
merged_pr = len([pr for pr in pr_data if pr['state'] == 'merged'])
closed_pr = len([pr for pr in pr_data if pr['state'] == 'closed' and not pr['merged']])

# 识别僵尸PR
zombie_prs = []
for pr in pr_data:
    if pr['state'] == 'open':
        days_open = (datetime.now() - pr['created_at'].replace(tzinfo=None)).days
        if days_open > days_threshold:
            zombie_prs.append(pr)

return {
    'total_pr': total_pr,
    'merged_pr': merged_pr,
    'closed_pr': closed_pr,
    'merge_rate': (merged_pr / total_pr) * 100 if total_pr > 0 else 0,
    'zombie_pr_count': len(zombie_prs),
```

```
'zombie_pr_rate': (len(zombie_prs) / total_pr) * 100 if total_pr > 0 else 0
}
```

分析覆盖 23 个 PR，核心指标如下：总 PR 数量 23 个，合并率 69.57

指标表明项目开发流程健康，PR 合并率高、审查效率良好，僵尸 PR 率控制在较低水平。

3.4.3 GitHub Actions workflow 分析

通过 GitHubActionsAnalyzer 模块分析项目 CI/CD 流程，实现 workflow 安全性与效率全面评估：

```
class GitHubActionsAnalyzer:
    def __init__(self, repo_owner, repo_name, output_dir, token=None):
        self.repo_owner = repo_owner
        self.repo_name = repo_name
        self.output_dir = Path(output_dir)
        self.github = Github(token) if token else Github()
        self.repo = self.github.get_repo(f"{repo_owner}/{repo_name}")

    def get_workflows(self):
        """获取所有Actions工作流"""
        workflows = []
        try:
            for workflow in self.repo.get_workflows():
                workflows.append({
                    'id': workflow.id,
                    'name': workflow.name,
                    'path': workflow.path,
                    'state': workflow.state
                })
        except Exception as e:
            print(f"获取工作流失败: {e}")

        return workflows
```

```
def analyze_workflow_security(self, workflows):
    """分析 workflow 安全性"""
    security_issues = []

    for workflow in workflows:
        try:
            file_content = self.repo.get_contents(workflow['path'])
            content = file_content.decoded_content.decode('utf-8')

            # 检测安全风险模式
            risks = self._check_security_patterns(content)
            if risks:
                security_issues.append({
                    'workflow': workflow['name'],
                    'risks': risks
                })

        except Exception as e:
            print(f"分析 workflow {workflow['name']} 失败: {e}")

    return security_issues

def _check_security_patterns(self, content):
    """检查 workflow 安全风险模式"""
    risks = []
    dangerous_patterns = [
        r'curl.*\\s*bash', # curl | bash 风险
        r'wget.*\\s*bash', # wget | bash 风险
        r'uses:\\s*actions/..*v\\d', # 使用版本标签而非 commit hash
    ]

    for pattern in dangerous_patterns:
```

```
if re.search(pattern, content, re.IGNORECASE):  
    risks.append(f"检测到危险模式: {pattern}")
```

```
return risks
```

分析结果：分析 workflow 数量 5 个，workflow 运行记录 14 次，安全问题数量 0 个。

项目 CI/CD 安全性表现良好，未发现 Actions workflow 安全问题，自动化流程配置安全可靠。

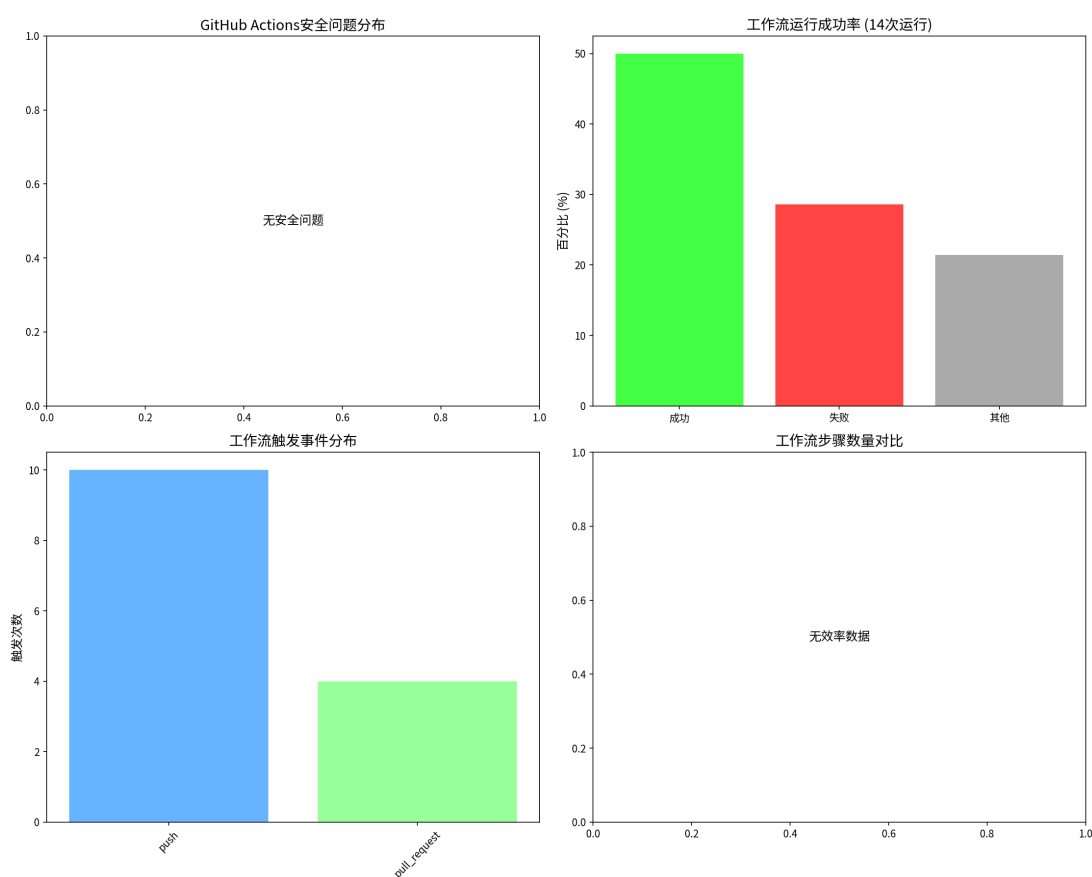


图 3.5 GitHub Actions 工作流分析结果

3.5 综合评估

基于深度分析结果，COMTool 项目代码质量整体表现良好：在代码复杂度方面控制良好，维护成本较低；在依赖安全方面无已知漏洞依赖，供应链管理优秀；在代码安全方面存在部分中低风险漏洞，需重点关注 5 个高风险漏洞；在社区健康度方面 PR 合并

率高，审查效率良好，开发流程健康；在 CI/CD 安全方面 workflow 配置安全，无已知安全问题。

建议开发团队重点处理 Bandit 扫描发现的高风险漏洞及 GitHub 安全 issue，持续优化代码质量与安全性。

结 论

本文设计并实现了一套多维度的软件仓库分析工具，通过整合 Git 版本控制数据、代码静态分析以及 GitHub 平台元数据，构建了从代码底层到社区协作层的全景质量评估体系。

在版本控制分析方面，工具结合可视化技术与统计模型，清晰重构了 COMTool 项目的演进轨迹。通过对提交频率、代码行数变动及文件热点的挖掘，直观展示了团队的开发节奏与协作分布，验证了项目具有稳定的维护模式。无论是宏观的月度趋势还是微观的周迭代节奏，都为管理者把握项目现状提供了量化依据。

在代码质量与安全审计方面，本研究实施了深度的静态代码扫描。复杂度分析表明项目主体结构得益于良好的模块化设计而保持了较高的可维护性。供应链审计通过全链路解析未发现已知的高危依赖漏洞，证明了版本管理的有效性。然而，基于 AST 的安全扫描检出了 45 个潜在安全隐患（其中包含 5 个高风险项），这为后续的代码加固与技术债务偿还提供了精确的导向。

在 GitHub 生态数据挖掘方面，工具量化了项目的社区响应能力与 CI/CD 流程健康度。通过对 Issue、Pull Request 及 GitHub Actions 的全量分析，不仅确认了自动化 workflows 的安全性，还识别出了关键的安全 Issue，体现了“代码-社区”联动分析的价值。特别是对 PR 处理周期的量化，揭示了团队协作效率优化的潜在空间。

本工具采用了高内聚低耦合的模块化架构，覆盖了全面性、自动化与可视化三大特征，实现了从数据清洗、指标计算到报告生成的全流程自动化。未来的工作将着眼于拓展多语言支持、引入大语言模型进行语义级代码理解，以及构建实时的质量监控仪表盘，以进一步提升工具在复杂软件工程场景下的适用性与智能化水平。

通过持续改进和优化，本工具将成为软件开发团队进行项目质量管理和安全评估的重要助手。

附录 A 原始分析数据摘录

为了佐证正文中的分析结论，本附录摘录了部分自动化工具生成的原始分析报告。这些数据直接来源于工具链的输出，未进行人工修饰。

A.1 Bandit 安全扫描报告（节选）

以下是 bandit 工具对源代码进行 AST 静态扫描后输出的部分高风险与中风险问题记录。报告详细列出了漏洞类型、严重程度及对应的代码位置。

```
Run started:2026-02-09 03:44:55.548818+00:00
```

```
Test results:
```

```
>> Issue: [B606:start_process_with_no_shell] Starting a process without a shell.
```

```
Severity: Low Confidence: Medium
```

```
CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
```

```
Location: repo/COMTool/Main.py:16:4
```

```
15         log.i("Restarting..., comand: {} {} {}".format(python, python, *sys.arg
```

```
16         os.execl(python, python, * sys.argv)
```

```
-----  
>> Issue: [B113:request_without_timeout] Call to requests without timeout
```

```
Severity: Medium Confidence: Low
```

```
CWE: CWE-400 (https://cwe.mitre.org/data/definitions/400.html)
```

```
Location: repo/COMTool/autoUpdate.py:25:19
```

```
24         try:
```

```
25             page = requests.get(self.releaseApiUrl)
```

A.2 Radon 圈复杂度分析（节选）

以下是 radon 工具生成的圈复杂度 (Cyclomatic Complexity) 原始数据。每行显示了文件路径、类型 (M=Method, C=Class, F=Function)、代码行号、名称以及复杂度评分 (A 为最优, F 为最差)。

```
repo/COMTool/Combobox.py
```

```
M 23:4 ComboBox._showPopup - A (3)
```

```
C 5:0 ComboBox - A (2)
M 9:4 ComboBox.__init__ - A (1)
M 15:4 ComboBox.mouseReleaseEvent - A (1)
```

repo/COMTool/Main.py

```
F 10:0 restart_program - A (1)
```

repo/COMTool/autoUpdate.py

```
M 21:4 AutoUpdate.checkUpdate_github - B (7)
C 10:0 AutoUpdate - A (4)
M 50:4 AutoUpdate.checkUpdate_neucrack - A (4)
M 14:4 AutoUpdate.detectNewVersion - A (2)
M 72:4 AutoUpdate.decodeTag - A (2)
```

A.3 Pylint 代码风格检查（节选）

以下是 pylint 对代码规范性检查的输出样本。主要包含了命名规范（Snake Case）、文档字符串缺失（Missing docstring）以及导入错误等问题。

```
***** Module COMTool
```

```
repo/COMTool/__init__.py:1:0: C0103: Module name "COMTool" doesn't conform to
snake_case naming style (invalid-name)
```

```
***** Module COMTool.Combobox
```

```
repo/COMTool/Combobox.py:1:0: C0103: Module name "Combobox" doesn't conform to
snake_case naming style (invalid-name)
```

```
repo/COMTool/Combobox.py:5:0: C0115: Missing class docstring (missing-class-docstring)
```

```
repo/COMTool/Combobox.py:11:8: C0103: Variable name "listView" doesn't conform to
snake_case naming style (invalid-name)
```

```
repo/COMTool/Combobox.py:15:4: C0103: Method name "mouseReleaseEvent" doesn't conform
to snake_case naming style (invalid-name)
```

```
repo/COMTool/Combobox.py:18:4: C0103: Method name "showPopup" doesn't conform to
snake_case naming style (invalid-name)
```

A.4 PR 分析报告

本节展示了针对 Neutree/COMTool 仓库的 PR 深度分析报告。

仓库名称: Neutree/COMTool 分析时间: 2026-02-09 11:46:32 僵尸 PR 阈值: 7 天 异常值过滤阈值: 30 天

核心指标汇总

指标	数值
总 PR 数	23
合入 PR 数	16
拒绝 PR 数	5
未处理 PR 数	2
PR 合入率	69.57%
PR 拒绝率	21.74%
僵尸 PR 数	2
僵尸 PR 占比	8.7%
平均审核时长 (小时)	7.86
中位数审核时长 (小时)	1.58
过滤异常值后审核时长	7.86
平均评论数	1.65
平均提交次数	1.57
平均新增代码	138.78 行
平均删除代码	73.78 行
平均修改文件数	2.65 个
平均生命周期 (天)	0.9

关键结论

- (1) **PR 处理效率:** 仓库 PR 合入率为 69.57%，拒绝率为 21.74%，反映代码评审的严格程度。正常的拒绝率可能表明评审标准较为严格。
- (2) **僵尸 PR 情况:** 僵尸 PR 占比 8.7%，正常的比例表明需要关注 PR 处理效率。
- (3) **审核效率:** 平均审核时长 7.86 小时，中位数为 1.58 小时。过滤异常值后为 7.86 小时，正常的时长可能影响开发效率。
- (4) **代码质量:** 平均每次 PR 新增代码 138.78 行，删除代码 73.78 行，修改文件 2.65

个。较小规模的修改可能增加评审难度。

- (5) **PR 生命周期**：关闭 PR 的平均生命周期为 0.9 天，正常的生命周期可能表明评审流程需要优化。
- (6) **协作模式**：PR 的平均评论数为 1.65 条，较少的评论表明团队协作程度较高。
- (7) **提交习惯**：平均每次 PR 提交次数为 1.57 次，正常的提交次数可能表明开发过程较为迭代。

详细分析

PR 作者分布

作者	PR 数量	占比
YI-	2	8.7%
junchao98	2	8.7%
e71828	2	8.7%
lhdjply	2	8.7%
taotieren	2	8.7%
dependabot[bot]	2	8.7%
taorye	2	8.7%
wirano	1	4.3%
sunzigang	1	4.3%
kaidegit	1	4.3%

PR 标签分布

标签	PR 数量	占比
无标签	21	91.3%
dependencies	2	8.7%

PR 分支分析

目标分支分布

分支	PR 数量
master	23

来源分支分布

分支	PR 数量
master	13
aur	2
feat/record	1
fix/protocols	1
makensis	1
develop/savetimesamp	1
GImDX-patch-1	1
fix-badge	1
dependabot/pip/urllib3-1.26.5	1
dependabot/pip/urllib3-1.25.8	1

PR 生命周期分布

生命周期	PR 数量
<1 天	19
1-3 天	1
3-7 天	0
7-14 天	0
14-30 天	1
30-60 天	0
>60 天	0

报告生成时间：2026-02-09 11:46:32